Chapter 1 — What Are High Energy Materials?

Python Code Examples · Jupyter Notebook Edition

Topics covered in 7 runnable blocks:

Block	Topic
0	Install & import all libraries
1	Classify energetic materials
2	Structure data from PubChem
3	Molecular properties with RDKit + draw structures
4	Oxygen balance calculation
5	Kamlet-Jacobs detonation estimates
6	Plot: sensitivity vs detonation velocity
7	ML classifier: primary vs secondary explosive

> **Run each cell in order from top to bottom (Shift+Enter).**

Block 0 · Install & Import Libraries

Run this once. All later blocks depend on these imports.

In [1]:
```
# ── Install (uncomment if running for the first time) ──────────
# !pip install rdkit pubchempy pandas matplotlib scikit-learn

# ── Standard library ──────────────────────────────────────────
import re, math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.lines import Line2D

# ── RDKit ─────────────────────────────────────────────────────
from rdkit import Chem
from rdkit.Chem import Descriptors, rdMolDescriptors, Draw
from rdkit.Chem.Draw import MolsToGridImage

# ── Scikit-learn ──────────────────────────────────────────────
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

# ── Jupyter display helpers ───────────────────────────────────
from IPython.display import display, HTML
```

```
%matplotlib inline
plt.rcParams['figure.dpi'] = 120

print('✓ All libraries loaded successfully')
```

```
C:\Users\admin\AppData\Roaming\Python\Python39\site-packages\pandas\core\computation\expressions.py:21: UserWarning: Pandas requires version '2.8.4' or newer of 'numexpr' (version '2.8.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
C:\Users\admin\AppData\Roaming\Python\Python39\site-packages\pandas\core\arrays\masked.py:61: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.4' currently installed).
  from pandas.core import (
✓ All libraries loaded successfully
```

Block 1 · Classification of Energetic Materials

Energetic materials are divided into **three main classes** based on how they release energy.

Class	How energy is released	Example use
Explosives (primary)	Very fast — triggered by small stimulus	Detonators
Explosives (secondary)	Fast — needs a primary to start	Warheads, mining
Propellants	Controlled burn — sustained push	Rockets, guns
Pyrotechnics	Burn for light/smoke/sound	Fireworks, flares

In [2]:
```
# ── Energetic material classification ──────────────────────────

em_classes = {
    "Primary Explosives":   ["Lead Azide", "Mercury Fulminate", "Silver Azide"],
    "Secondary Explosives": ["TNT", "RDX", "HMX", "PETN", "CL-20", "TATB"],
    "Propellants":          ["Nitrocellulose", "Ammonium Perchlorate (APCP)", "HTPE
    "Pyrotechnics":         ["Black Powder", "Thermite", "Strontium Nitrate Mix"],
    "Insensitive Explosives": ["TATB", "NTO", "FOX-7", "DNAN"],
}

# Build a styled HTML table for Jupyter
rows = ""
colors_map = {
    "Primary Explosives":     "#ffe0e0",
    "Secondary Explosives":   "#fff3cd",
    "Propellants":            "#d4edda",
    "Pyrotechnics":           "#cce5ff",
    "Insensitive Explosives": "#e2d9f3",
}
for category, compounds in em_classes.items():
    bg = colors_map.get(category, "#f8f9fa")
    rows += (f'<tr style="background:{bg}">'
             f'<td><b>{category}</b></td>'
             f'<td>{", ".join(compounds)}</td></tr>')

html = ("<table style='border-collapse:collapse;width:100%;font-size:14px'>"
        "<tr style='background:#333;color:#fff'><th style='padding:8px;text-align:l
        "<th style='padding:8px;text-align:left'>Examples</th></tr>"
        + rows + "</table>")
display(HTML(html))
```

Class	Examples
Primary Explosives	Lead Azide, Mercury Fulminate, Silver Azide
Secondary Explosives	TNT, RDX, HMX, PETN, CL-20, TATB
Propellants	Nitrocellulose, Ammonium Perchlorate (APCP), HTPB
Pyrotechnics	Black Powder, Thermite, Strontium Nitrate Mix
Insensitive Explosives	TATB, NTO, FOX-7, DNAN

Block 2 · Structure Data from PubChem

Each compound has a **SMILES string** that encodes its structure as text.
PubChem (pubchem.ncbi.nlm.nih.gov) is the largest free chemical database.

> **Try it live:** uncomment the `pubchempy` lines to fetch real-time data.

In [4]:
```
# ── PubChem structure data ──────────────────────────────────
# Preloaded from PubChem (same values the API returns).
# To fetch live: pip install pubchempy  then uncomment the block below.

# --- LIVE FETCH (requires internet) ---
# import pubchempy as pcp
# def fetch(name):
#     r = pcp.get_compounds(name, 'name')
#     c = r[0] if r else None
#     return {'Name': name,
#             'Formula': c.molecular_formula if c else 'N/A',
#             'MW (g/mol)': round(c.molecular_weight, 2) if c else 'N/A',
#             'SMILES': c.isomeric_smiles if c else 'N/A'}
# records = [fetch(n) for n in ["TNT","RDX","HMX","PETN","CL-20","TATB","NTO"]]

# --- PRELOADED DATA (works offline) ---
records = [
    {"Name": "TNT",  "Formula": "C7H5N3O6",   "MW (g/mol)": 227.13,
     "SMILES": "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]"},
    {"Name": "RDX",  "Formula": "C3H6N6O6",   "MW (g/mol)": 222.12,
     "SMILES": "O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1"},
    {"Name": "HMX",  "Formula": "C4H8N8O8",   "MW (g/mol)": 296.16,
     "SMILES": "O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C1"},
    {"Name": "PETN", "Formula": "C5H8N4O12",  "MW (g/mol)": 316.14,
     "SMILES": "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]"},
    {"Name": "CL-20","Formula": "C6H6N12O12", "MW (g/mol)": 438.19,
     "SMILES": "O=[N+]([O-])N1CN2CN([N+](=O)[O-])CN1[N+](=O)[O-]"},
    {"Name": "TATB", "Formula": "C6H6N6O6",   "MW (g/mol)": 258.15,
     "SMILES": "Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])c(N)c1[N+](=O)[O-]"},
    {"Name": "NTO",  "Formula": "C2H2N4O3",   "MW (g/mol)": 130.06,
     "SMILES": "O=[N+]([O-])c1n[nH]c(=O)[nH]1"},
]

df_pubchem = pd.DataFrame(records)
display(df_pubchem)
'''
```

```
display(df_pubchem.style
        .set_caption("PubChem Data for Common Energetic Compounds")
        .set_table_styles([{'selector': 'caption',
                            'props': [('font-size','14px'),('font-weight','bold')]}])
        .hide(axis='index'))
'''
```

	Name	Formula	MW (g/mol)	SMILES
0	TNT	C7H5N3O6	227.13	Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]
1	RDX	C3H6N6O6	222.12	O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1
2	HMX	C4H8N8O8	296.16	O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])...
3	PETN	C5H8N4O12	316.14	C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O...
4	CL-20	C6H6N12O12	438.19	O=[N+]([O-])N1CN2CN([N+](=O)[O-])CN1[N+](=O)[O-]
5	TATB	C6H6N6O6	258.15	Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])c(N)c1[N+...
6	NTO	C2H2N4O3	130.06	O=[N+]([O-])c1n[nH]c(=O)[nH]1

Out[4]:
```
'\ndisplay(df_pubchem.style\n        .set_caption("PubChem Data for Common Energet
ic Compounds")\n        .set_table_styles([{\'selector\': \'caption\',\n
\'props\': [(\'font-size\',\'14px\'),(\'font-weight\',\'bold\')]}])\n        .hide
(axis=\'index\'))\n'
```

Block 3 · Molecular Properties with RDKit + Structure Images

RDKit converts a SMILES string into a **molecule object** from which we can compute:

- Molecular weight
- Number of nitro groups ($-NO_2$)
- Number of rings
- H-bond donors / acceptors
- Rotatable bonds

It can also **draw the structure** directly in the notebook.

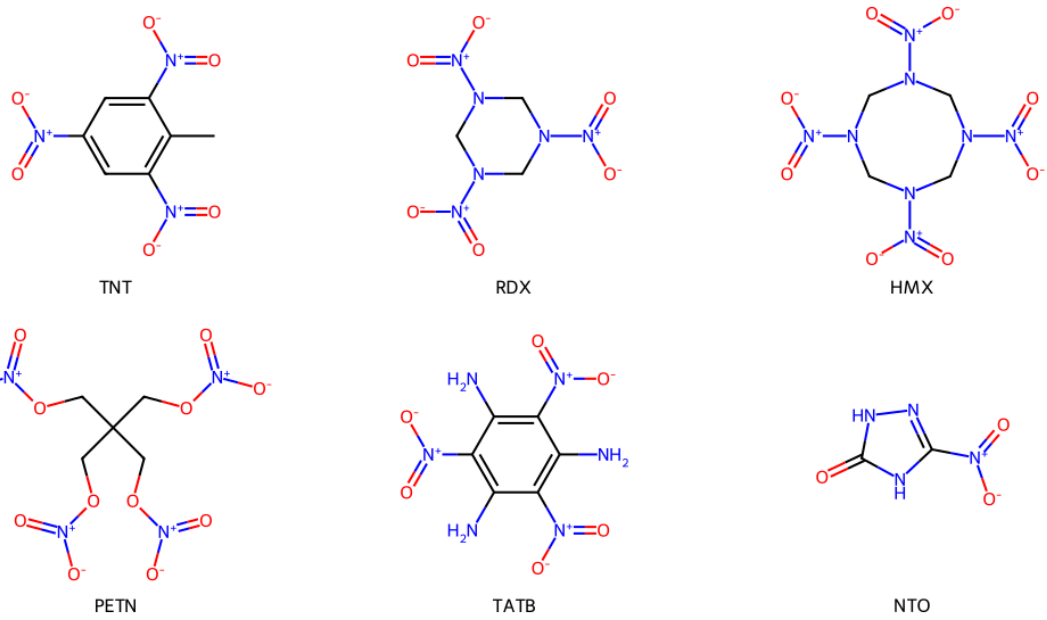
In [5]:
```
# ── 3a: Compute molecular properties for each compound ──────────

smiles_dict = {
    "TNT":  "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]",
    "RDX":  "O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1",
    "HMX":  "O=[N+]([O-])N1CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C1",
    "PETN": "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]",
    "TATB": "Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])c(N)c1[N+](=O)[O-]",
    "NTO":  "O=[N+]([O-])c1n[nH]c(=O)[nH]1",
}

def rdkit_props(name, smi):
    mol = Chem.MolFromSmiles(smi)
    if mol is None:
        return None
    return {
        "Name":        name,
        "MW":          round(Descriptors.MolWt(mol), 2),
```

```
        "#NO2 groups": smi.count("[N+](=O)[O-]"),
        "#Rings":      rdMolDescriptors.CalcNumRings(mol),
        "HBD":         rdMolDescriptors.CalcNumHBD(mol),
        "HBA":         rdMolDescriptors.CalcNumHBA(mol),
        "RotBonds":    rdMolDescriptors.CalcNumRotatableBonds(mol),
        "TPSA":        round(Descriptors.TPSA(mol), 1),
    }

props = [rdkit_props(n, s) for n, s in smiles_dict.items()]
props = [p for p in props if p]          # remove None (bad SMILES)
df_props = pd.DataFrame(props)
display(df_props)
"""
display(df_props.style
        .set_caption("RDKit Molecular Properties")
        .background_gradient(subset=["MW","#NO2 groups","#Rings"], cmap="YlOrRd")
        .hide(axis='index')
        .set_table_styles([{'selector':'caption',
                            'props':[('font-size','14px'),('font-weight','bold')]}]
"""
```

	Name	MW	#NO2 groups	#Rings	HBD	HBA	RotBonds	TPSA
0	TNT	227.13	3	1	0	6	3	129.4
1	RDX	222.12	2	1	0	6	3	139.1
2	HMX	296.16	3	1	0	8	4	185.5
3	PETN	316.13	4	0	0	12	12	209.5
4	TATB	258.15	3	1	3	9	3	207.5
5	NTO	130.06	0	1	2	4	1	104.7

Out[5]:
```
'\ndisplay(df_props.style\n        .set_caption("RDKit Molecular Properties")\n
.background_gradient(subset=["MW","#NO2 groups","#Rings"], cmap="YlOrRd")\n
.hide(axis=\'index\')\n        .set_table_styles([{\'selector\':\'caption\',\n
\'props\':[(\'font-size\',\'14px\'),(\'font-weight\',\'bold\')]}]))\n'
```

In [6]:
```
# ── 3b: Draw molecule structures as a grid ──────────────────────

mols   = [Chem.MolFromSmiles(s) for s in smiles_dict.values()]
labels = list(smiles_dict.keys())

img = MolsToGridImage(
    mols,
    molsPerRow=3,
    subImgSize=(350, 280),
    legends=labels,
)
display(img)   # shows the image inline in Jupyter
```



Block 4 · Oxygen Balance (OB%)

Oxygen balance tells you whether a compound has enough oxygen to combust completely.

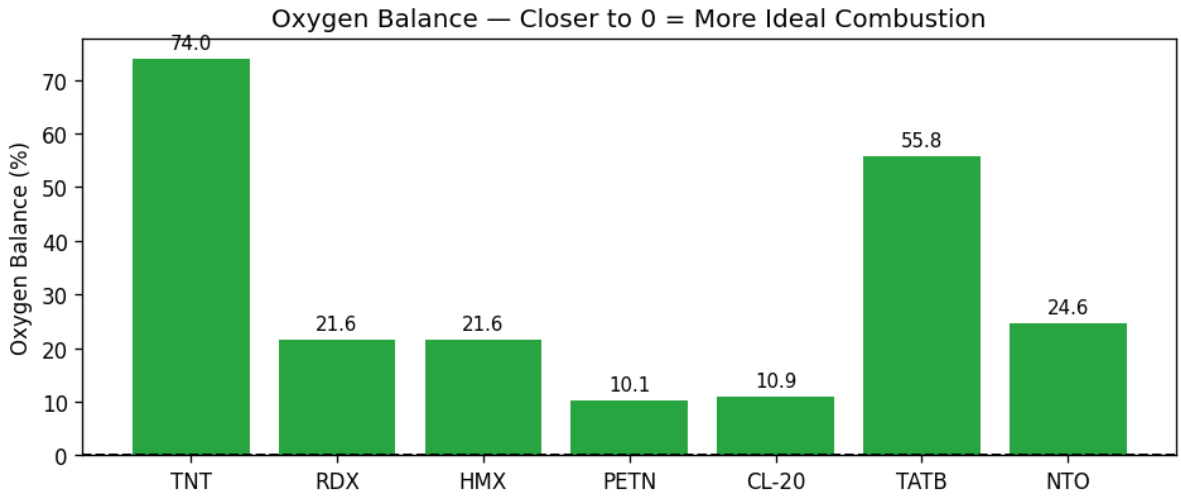
$$OB\% = \frac{1600}{MW} \times \left(2C + \frac{H}{2} - O\right)$$

OB%	Meaning
≈ 0	Ideal — complete combustion, maximum energy
Negative	Fuel-rich — some carbon becomes soot
Positive	Oxygen-rich — not all fuel is consumed

In [9]:
```python
# ── Oxygen balance calculation ──────────────────────────────

def parse_formula(formula):
    """Return {C, H, N, O} counts from a molecular formula string."""
    counts = {}
    for elem in ["C", "H", "N", "O"]:
        m = re.search(rf"{elem}(\d*)", formula)
        counts[elem] = int(m.group(1)) if m and m.group(1) else (1 if m else 0)
    return counts

def oxygen_balance(formula, mw):
    e = parse_formula(formula)
    return round((1600 / mw) * (2*e["C"] + e["H"]/2 - e["O"]), 2)

# Data: (molecular formula, MW, crystal density g/cc)
em_data = {
    "TNT":   ("C7H5N3O6",   227.13, 1.654),
    "RDX":   ("C3H6N6O6",   222.12, 1.820),
    "HMX":   ("C4H8N8O8",   296.16, 1.905),
    "PETN":  ("C5H8N4O12",  316.14, 1.778),
    "CL-20": ("C6H6N12O12", 438.19, 2.044),
    "TATB":  ("C6H6N6O6",   258.15, 1.939),
    "NTO":   ("C2H2N4O3",   130.06, 1.930),
}
```

```
rows = []
for name, (formula, mw, rho) in em_data.items():
    ob = oxygen_balance(formula, mw)
    rows.append({"Compound": name, "Formula": formula,
                 "MW (g/mol)": mw, "Density (g/cc)": rho, "OB%": ob})

df_ob = pd.DataFrame(rows)

def color_ob(val):
    """Color: green near 0, red when very negative or positive."""
    if   val > 5:   return 'background-color: #c3e6cb'
    elif val > -20: return 'background-color: #fff3cd'
    else:           return 'background-color: #f5c6cb'

display(df_ob)

"""
display(df_ob.style
        .applymap(color_ob, subset=["OB%"])
        .set_caption("Oxygen Balance for Common Energetic Compounds")
        .hide(axis='index')
        .set_table_styles([{'selector':'caption',
                            'props':[('font-size','14px'),('font-weight','bold')]}])
"""


# Quick bar chart of OB%
fig, ax = plt.subplots(figsize=(8, 3.5))
bars = ax.bar(df_ob["Compound"], df_ob["OB%"],
              color=["#28a745" if v > -20 else "#dc3545" for v in df_ob["OB%"]])
ax.axhline(0, color='black', linewidth=1.2, linestyle='--')
ax.set_ylabel("Oxygen Balance (%)")
ax.set_title("Oxygen Balance — Closer to 0 = More Ideal Combustion")
ax.bar_label(bars, fmt='%.1f', padding=3, fontsize=9)
plt.tight_layout()
plt.show()
```

	Compound	Formula	MW (g/mol)	Density (g/cc)	OB%
0	TNT	C7H5N3O6	227.13	1.654	73.97
1	RDX	C3H6N6O6	222.12	1.820	21.61
2	HMX	C4H8N8O8	296.16	1.905	21.61
3	PETN	C5H8N4O12	316.14	1.778	10.12
4	CL-20	C6H6N12O12	438.19	2.044	10.95
5	TATB	C6H6N6O6	258.15	1.939	55.78
6	NTO	C2H2N4O3	130.06	1.930	24.60



Block 5 · Kamlet-Jacobs Detonation Estimates

The **Kamlet-Jacobs (KJ) equations** are the most widely used empirical formulas for predicting detonation velocity (D) and detonation pressure (P) from four inputs:

Symbol	Meaning	Units
N	Moles of gas per gram of explosive	mol/g
M	Average MW of gaseous products	g/mol
Q	Heat of explosion	cal/g
ρ	Crystal density	g/cm³

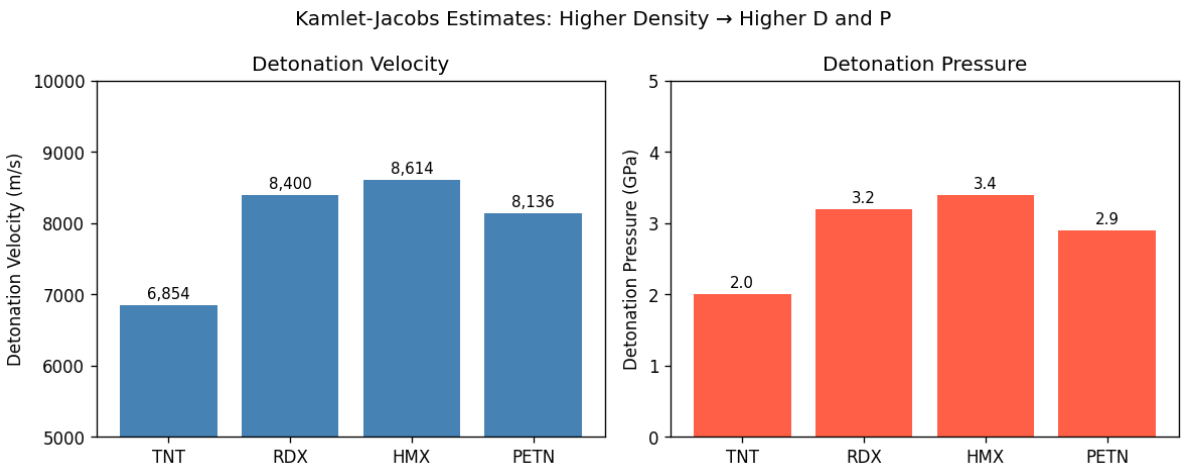
$$D\ (\mathrm{km/s}) = 1.01\ \sqrt{\phi}\,(1 + 1.30\,\rho)$$

$$P\ (\mathrm{Mbar}) = 1.558 \times 10^{-3}\,\rho^2\,\phi \quad \text{where} \quad \phi = N\sqrt{MQ}$$

In [10]:
```
# ── Kamlet-Jacobs detonation velocity & pressure ──────────────────

def kamlet_jacobs(N, M, Q, rho):
    """Returns (D in m/s, P in GPa)."""
    phi = N * math.sqrt(M * Q)
    D_kms  = 1.01 * math.sqrt(phi) * (1 + 1.30 * rho)   # km/s
    P_mbar = 1.558e-3 * rho**2 * phi                     # Mbar
    return round(D_kms * 1000), round(P_mbar * 100, 1)   # m/s, GPa

# Kamlet-Jacobs parameters (literature values for CHNO explosives)
#                  N          M       Q       rho
kj_params = {
    "TNT":  (0.01987, 50.51, 1080.0, 1.654),
    "RDX":  (0.02987, 27.67, 1510.0, 1.820),
    "HMX":  (0.02975, 27.61, 1482.0, 1.905),
    "PETN": (0.02865, 28.26, 1510.0, 1.778),
}

kj_rows = []
for name, (N, M, Q, rho) in kj_params.items():
    D, P = kamlet_jacobs(N, M, Q, rho)
    kj_rows.append({"Compound": name, "ρ (g/cc)": rho,
                    "Q (cal/g)": Q, "D (m/s)": D, "P (GPa)": P})
```

```
df_kj = pd.DataFrame(kj_rows)

display(df_kj)

"""
display(df_kj.style
        .background_gradient(subset=["D (m/s)","P (GPa)"], cmap="Oranges")
        .set_caption("Kamlet-Jacobs Detonation Estimates")
        .hide(axis='index')
        .set_table_styles([{'selector':'caption',
                            'props':[('font-size','14px'),('font-weight','bold')]}]
"""


# Bar chart comparing D and P
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.bar(df_kj["Compound"], df_kj["D (m/s)"], color="steelblue")
ax1.set_ylabel("Detonation Velocity (m/s)")
ax1.set_title("Detonation Velocity")
ax1.set_ylim(5000, 10000)
for bar, val in zip(ax1.patches, df_kj["D (m/s)"]):
    ax1.text(bar.get_x()+bar.get_width()/2, bar.get_height()+50,
             f"{val:,}", ha='center', va='bottom', fontsize=9)

ax2.bar(df_kj["Compound"], df_kj["P (GPa)"], color="tomato")
ax2.set_ylabel("Detonation Pressure (GPa)")
ax2.set_title("Detonation Pressure")
ax2.set_ylim(0, 5)
for bar, val in zip(ax2.patches, df_kj["P (GPa)"]):
    ax2.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.05,
             f"{val}", ha='center', va='bottom', fontsize=9)

plt.suptitle("Kamlet-Jacobs Estimates: Higher Density → Higher D and P", fontsize=1
plt.tight_layout()
plt.show()
```

	Compound	ρ (g/cc)	Q (cal/g)	D (m/s)	P (GPa)
0	TNT	1.654	1080.0	6854	2.0
1	RDX	1.820	1510.0	8400	3.2
2	HMX	1.905	1482.0	8614	3.4
3	PETN	1.778	1510.0	8136	2.9



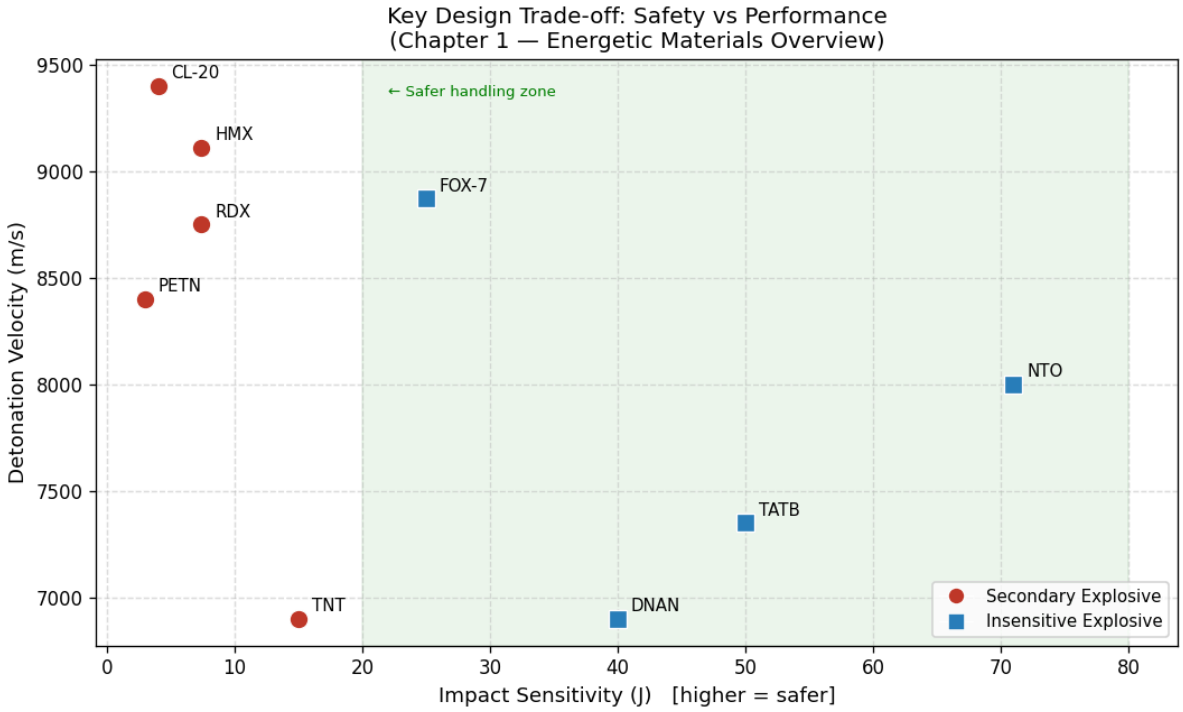
Block 6 · Plot: Sensitivity vs Detonation Velocity

This is the **core trade-off in energetic material design**:
compounds that detonate faster are usually more sensitive (more dangerous to handle).

- **Impact sensitivity (J):** higher = safer (harder to set off accidentally)
- **Detonation velocity (m/s):** higher = more powerful

The goal is to find materials in the **top-right corner** — high velocity AND high safety.

In [11]:
```
# ── Sensitivity vs Detonation Velocity scatter plot ──────────────

# Experimental data (literature values)
#              Impact sens (J)   Det. vel (m/s)   Type
exp_data = {
    "TNT":    (15.0, 6900, "Secondary"),
    "RDX":    ( 7.4, 8750, "Secondary"),
    "HMX":    ( 7.4, 9110, "Secondary"),
    "PETN":   ( 3.0, 8400, "Secondary"),
    "CL-20":  ( 4.0, 9400, "Secondary"),
    "TATB":   (50.0, 7350, "Insensitive"),
    "NTO":    (71.0, 8000, "Insensitive"),
    "DNAN":   (40.0, 6900, "Insensitive"),
    "FOX-7":  (25.0, 8870, "Insensitive"),
}

style = {"Secondary":   {"color": "#c0392b", "marker": "o", "size": 100},
         "Insensitive": {"color": "#2980b9", "marker": "s", "size": 100}}

fig, ax = plt.subplots(figsize=(9, 5.5))

for name, (sens, vel, typ) in exp_data.items():
    s = style[typ]
    ax.scatter(sens, vel, color=s["color"], marker=s["marker"],
               s=s["size"], zorder=4, edgecolors='white', linewidths=0.8)
    ax.annotate(name, (sens, vel),
                textcoords="offset points", xytext=(7, 4), fontsize=9)

# Ideal zone annotation
ax.axvspan(20, 80, alpha=0.07, color='green', label='_nolegend_')
ax.text(22, 9350, "← Safer handling zone", fontsize=8, color='green')

# Legend
legend_elems = [
    Line2D([0],[0], marker='o', color='w', markerfacecolor='#c0392b',
           markersize=9, label='Secondary Explosive'),
    Line2D([0],[0], marker='s', color='w', markerfacecolor='#2980b9',
           markersize=9, label='Insensitive Explosive'),
]
ax.legend(handles=legend_elems, loc='lower right', fontsize=9)

ax.set_xlabel("Impact Sensitivity (J)   [higher = safer]", fontsize=11)
ax.set_ylabel("Detonation Velocity (m/s)", fontsize=11)
ax.set_title("Key Design Trade-off: Safety vs Performance\n"
             "(Chapter 1 — Energetic Materials Overview)", fontsize=12)
ax.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()

print("Observation: More powerful explosives (high D) tend to be more sensitive (lo
print("Insensitive materials trade some performance for safer handling.")
```




```
Observation: More powerful explosives (high D) tend to be more sensitive (low J).
Insensitive materials trade some performance for safer handling.
```

Block 7 · ML Classifier: Primary vs Secondary Explosive

We train a **Random Forest** classifier using four simple features:

Feature	Why it matters
`#NO2`	More nitro groups → secondary explosive
`OB%`	Oxygen balance pattern differs by class
`MW`	Secondary explosives tend to be heavier
`#Rings`	Ring structures are common in secondary explosives

This demonstrates the **basic ML pipeline**: data → train → evaluate → predict.

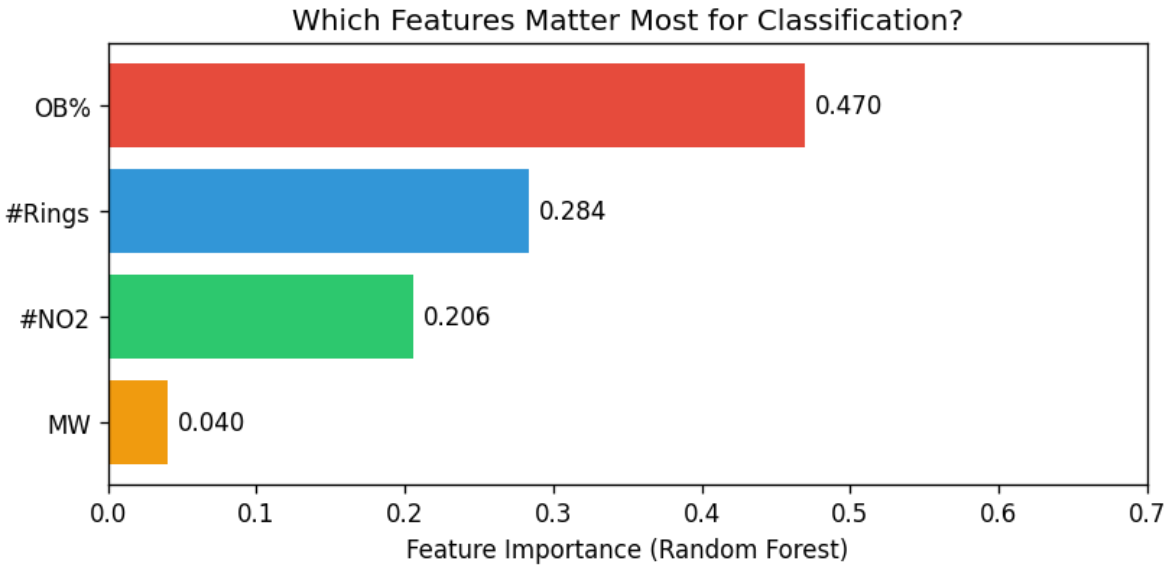
In [12]:
```
# ── 7a: Build dataset and train classifier ──────────────────────

# Columns: [#NO2, OB%,  MW,   #Rings, label]
# Label  :  0=Primary,  1=Secondary
raw = [
    # --- Primary explosives (sensitive, few nitro groups, no rings) ---
    [1, -3.2,  291, 0, 0, "Lead Azide"],
    [2,  2.0,  284, 0, 0, "Mercury Fulminate"],
    [1, -5.1,  170, 0, 0, "Silver Azide"],
    [2,  0.0,  165, 0, 0, "Lead Styphnate"],
    [1, -2.5,  305, 0, 0, "Copper Azide"],
    [1,  1.5,  248, 0, 0, "Diazodinitrophenol"],
    # --- Secondary explosives (less sensitive, more nitro, rings) ---
    [3, -74.0, 227, 1, 1, "TNT"],
    [3, -21.6, 222, 1, 1, "RDX"],
    [4, -21.6, 296, 1, 1, "HMX"],
    [4, -10.1, 316, 0, 1, "PETN"],
```

```
    [6, -11.0, 438, 2, 1, "CL-20"],
    [3, -56.3, 229, 1, 1, "Tetryl"],
    [3, -45.0, 258, 1, 1, "TATB"],
    [1, -24.6, 130, 1, 1, "NTO"],
]

feature_names = ["#NO2", "OB%", "MW", "#Rings"]
label_names   = ["Primary", "Secondary"]

X = np.array([r[:4]  for r in raw])
y = np.array([r[4]   for r in raw])
names_col = [r[5]    for r in raw]

X_train, X_test, y_train, y_test, names_train, names_test = train_test_split(
    X, y, names_col, test_size=0.30, random_state=42
)

clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("Training compounds:", names_train)
print("Test compounds    :", names_test)
print()
print(classification_report(y_test, y_pred,
      target_names=label_names, zero_division=0))
```

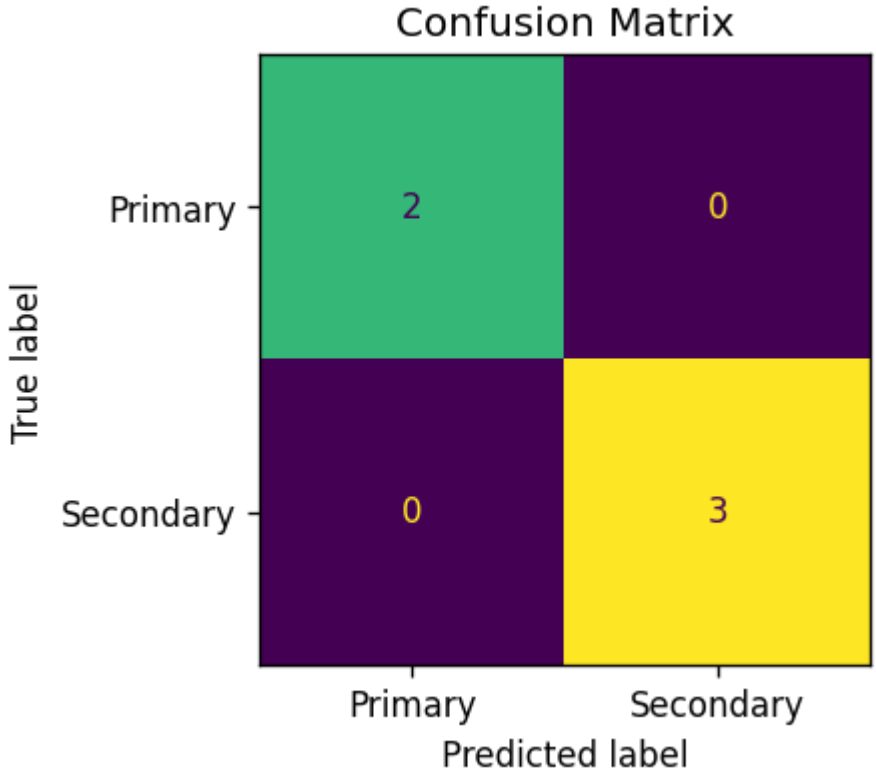
```
Training compounds: ['HMX', 'Silver Azide', 'Mercury Fulminate', 'NTO', 'Copper Az
ide', 'RDX', 'CL-20', 'Lead Styphnate', 'TNT']
Test compounds    : ['PETN', 'Tetryl', 'Lead Azide', 'TATB', 'Diazodinitrophenol']

              precision    recall  f1-score   support

     Primary       1.00      1.00      1.00         2
   Secondary       1.00      1.00      1.00         3

    accuracy                           1.00         5
   macro avg       1.00      1.00      1.00         5
weighted avg       1.00      1.00      1.00         5
```

In [13]:

```
# ── 7b: Feature importance chart ──────────────────────────────

importances = clf.feature_importances_
sorted_idx  = np.argsort(importances)[::-1]

fig, ax = plt.subplots(figsize=(7, 3.5))
bars = ax.barh(
    [feature_names[i] for i in sorted_idx],
    [importances[i]   for i in sorted_idx],
    color=["#e74c3c","#3498db","#2ecc71","#f39c12"][:len(sorted_idx)]
)
ax.bar_label(bars, fmt='%.3f', padding=4)
ax.set_xlabel("Feature Importance (Random Forest)")
ax.set_title("Which Features Matter Most for Classification?")
ax.set_xlim(0, 0.7)
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```



In [14]:
```
# ── 7c: Confusion matrix ──────────────────────────────────────

fig, ax = plt.subplots(figsize=(4, 3.5))
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred,
    display_labels=label_names,
    colorbar=False,
    ax=ax
)
ax.set_title("Confusion Matrix")
plt.tight_layout()
plt.show()
```



In [15]:
```
# ── 7d: Predict a NEW unknown compound ──────────────────────────

# Try changing these values and re-run the cell!
new_compounds = [
    {"name": "Unknown-A",  "#NO2": 3, "OB%": -30.0, "MW": 250, "#Rings": 1},
    {"name": "Unknown-B",  "#NO2": 1, "OB%":  -2.0, "MW": 180, "#Rings": 0},
```

```
    {"name": "Unknown-C",  "#NO2": 5, "OB%": -12.0, "MW": 380, "#Rings": 2},
]

print(f"{'Compound':<14} {'Predicted Class':<18} {'P(Primary)':>12} {'P(Secondary)'
print("-" * 60)
for c in new_compounds:
    x_new   = np.array([[c["#NO2"], c["OB%"], c["MW"], c["#Rings"]]])
    pred    = label_names[clf.predict(x_new)[0]]
    prob    = clf.predict_proba(x_new)[0]
    print(f"{c['name']:<14} {pred:<18} {prob[0]:>12.2f} {prob[1]:>14.2f}")
```

```
Compound       Predicted Class      P(Primary)   P(Secondary)
------------------------------------------------------------
Unknown-A      Secondary                  0.01           0.99
Unknown-B      Primary                    0.96           0.04
Unknown-C      Secondary                  0.15           0.85
```

Summary — What You Learned in Chapter 1

Block	Concept	Python tool
1	EM classification	Plain Python dict + HTML display
2	SMILES, formula, MW from PubChem	`pubchempy` / preloaded dict
3	Molecular properties + structure drawings	`rdkit`
4	Oxygen balance formula	`re` , `math` , `pandas`
5	Kamlet-Jacobs detonation estimates	`math` , `pandas` , `matplotlib`
6	Sensitivity vs performance trade-off	`matplotlib` scatter plot
7	Primary vs Secondary ML classifier	`scikit-learn` RandomForest

Next: Chapter 2 covers molecular descriptors and fingerprints in depth.

References & Tools

- RDKit: https://www.rdkit.org
- PubChem: https://pubchem.ncbi.nlm.nih.gov
- Scikit-learn: https://scikit-learn.org
- Kamlet & Jacobs (1968) *J. Chem. Phys.* 48(1)
- Cooper, P.W. *Explosives Engineering*, Wiley-VCH, 1996
- Karthikeyan & Vyas *Practical Chemoinformatics*, Springer, 2014

In [ ]: